ASSIGNMENT 1 – TASK 4

C Programming (CO1) | Use Macros Safely | 4 Marks

1. Aim

To define and use C preprocessor macros safely for repeated calculations, understand macro expansion, identify common macro pitfalls, and compare macros with static inline functions.

2. Source Code

```
#include <stdio.h>

#define TO_PAISE(rupees) ((rupees) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int main(void)
{
    int a = 4;
    int b = 7;

    printf("Square = %d\n", SQUARE(a + 1));
    printf("Maximum = %d\n", MAX(a, b));
    printf("Paise = %d\n", TO_PAISE(50));

    return 0;
}
```

The macros are fully parenthesised. This protects the intended order of evaluation when expressions are passed as arguments.

3. Output and Execution Evidence

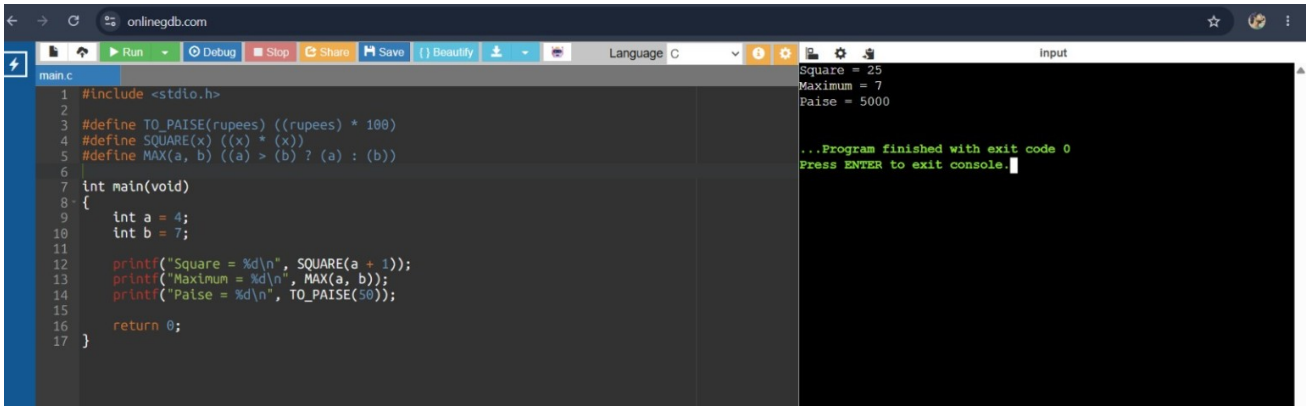


Figure 1. OnlineGDB execution screenshot supplied for Task 4.

```
Square = 25
Maximum = 7
Paise = 5000
```

Result: SQUARE(a + 1) calculates (4 + 1) × (4 + 1) = 25; MAX(4, 7) returns 7; and TO_PAISE(50) converts 50 rupees to 5000 paise.

4. How the Preprocessor Expands Macros

The preprocessor replaces each macro invocation with its replacement text before the compiler translates the C program. Use the following command to inspect the expanded source:

```
gcc -E main.c -o main.i
```

Representative expansions for the three calls in this program are:

```
SQUARE(a + 1)  ->  ((a + 1) * (a + 1))
MAX(a, b)      ->  ((a) > (b) ? (a) : (b))
TO_PAISE(50)   ->  ((50) * 100)
```

The full output from gcc -E also contains expanded system-header declarations, so it is much longer than these representative lines.

5. Common Macro Pitfalls

Pitfall	Explanation and safe practice
Missing parentheses	A macro written as `#define SQUARE(x) x*x` can expand `SQUARE(a+1)` to `a+1*a+1`. With a = 4, this evaluates to 9 rather than the intended 25. Use `((x) * (x))`.
Argument evaluated more than once	`MAX(a, b)` may evaluate one argument twice. For example, `MAX(i++, j)` can increment `i` more than once depending on the comparison result. Avoid side effects in macro arguments.
Type checking and debugging	Macros are text substitutions and do not provide normal function parameter type checking. Errors may be harder to trace in debugger output.

6. Alternative Concept: Static Inline Functions

```
static inline int square_int(int x)
{
    return x * x;
}

static inline int max_int(int a, int b)
{
    return (a > b) ? a : b;
}
```

Calls would be written as square_int(a + 1) and max_int(a, b). Inline functions are type-checked, evaluate each argument once, and are easier to debug. The compiler may inline them, but this is an optimization choice rather than a guarantee.

7. Time and Space Complexity

Approach	Time per use	Code-size / space consideration
Macro	O(1) for these arithmetic expressions.	Replacement code may be repeated at each of *k* uses, potentially increasing code size with *k*. No function-call frame is required for the macro itself.
Static inline function	O(1) for these arithmetic expressions; a call may have overhead if not inlined.	The compiler can inline calls or keep one function body. Usually easier to maintain; actual binary size depends on compiler optimization.

8. Conclusion

The program produces the expected output. Fully parenthesised macros reduce operator-precedence errors, but arguments should not contain side effects because macros can evaluate them more than once. For more complex calculations, static inline functions are generally safer due to type checking, single evaluation of arguments, and easier debugging.

Submission note: The supplied screenshot is included as execution evidence. If your instructor requires a separate terminal screenshot of gcc -E, run the command above and add that screenshot to the Task04 folder.